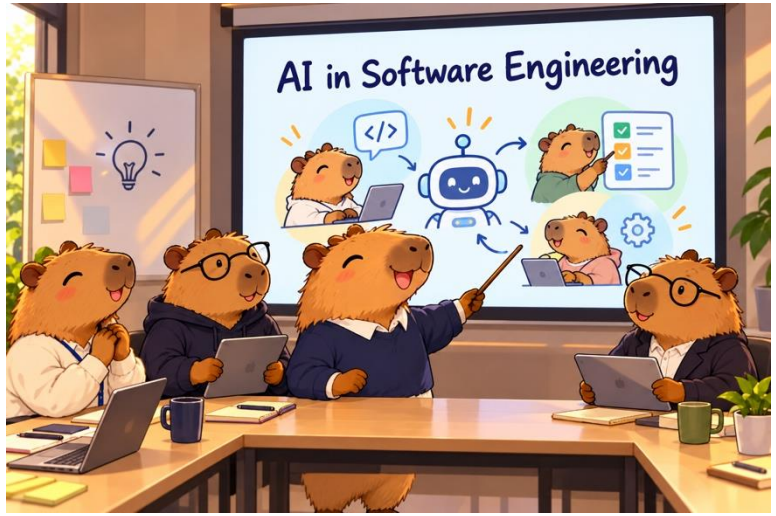
Software Project Management – IS212

Automated Testing #2 & Continuous Delivery (CD)

Announcements

Announcements – Topic Research & Sharing

- Thank you for selecting a topic to present in **Week 10**
  - *These are now "locked in" 🔒; no more changes*



- The first half of Week 10 will be a regular class; the topic presentations take place after the break
- Each team will have **7 minutes (strict)** to present, followed by up to 2 minutes of Q&A
- **Try to cover:** "what is it", "can you show it", "why does it matter", and "where does it break" – be critical
- Please see the briefing in eLearn for rubrics **(3%)**

Announcements – Automated Testing in the Exam

- Many students have asked what they need to know about automated testing for the final exam
- You will not need to write **complete** test classes **from scratch**
- You **should** be able to read, understand, and review test code (i.e., using `unittest` or `unittest.mock`), identify problems, and propose fixes or improvements
- We focus on concepts and correctness, not exact syntax
  - *For example, knowing what an assertion should check matters more than remembering the exact method name or capitalisation in `unittest`*
  - *We will provide some sample exam questions*

SMU Classification: Restricted

Recap

Week 6 Recap – Automated Testing #1; CI

- Your team's suite reports 100% statement coverage. What can you conclude – and what can't you?
- An agent wrote both `final_price()` and its tests; everything is green. Why might the requirement still be violated?
- A test passes on when run first but fails when run after another test. Which FIRST property does it break?
- In TDD, why must the test fail (Red) before you write any code?
- A mutant "survives" your test suite. What does that tell you?
- A teammate tries to merge while the test suite is failing. What should prevent the merge automatically?

Today: Automated Testing #2; CD

Objectives

On completing this class, you will be able to:

- Choose appropriate test doubles (stubs, mocks, fakes) and implement them with `unittest.mock`
- Write integration and end-to-end tests for a Flask application, and diagnose and prevent flaky tests
- Explain how a continuous delivery pipeline gates a release, and distinguish continuous delivery from continuous deployment
- Judge what doubled, generated, or "self-healed" tests actually prove

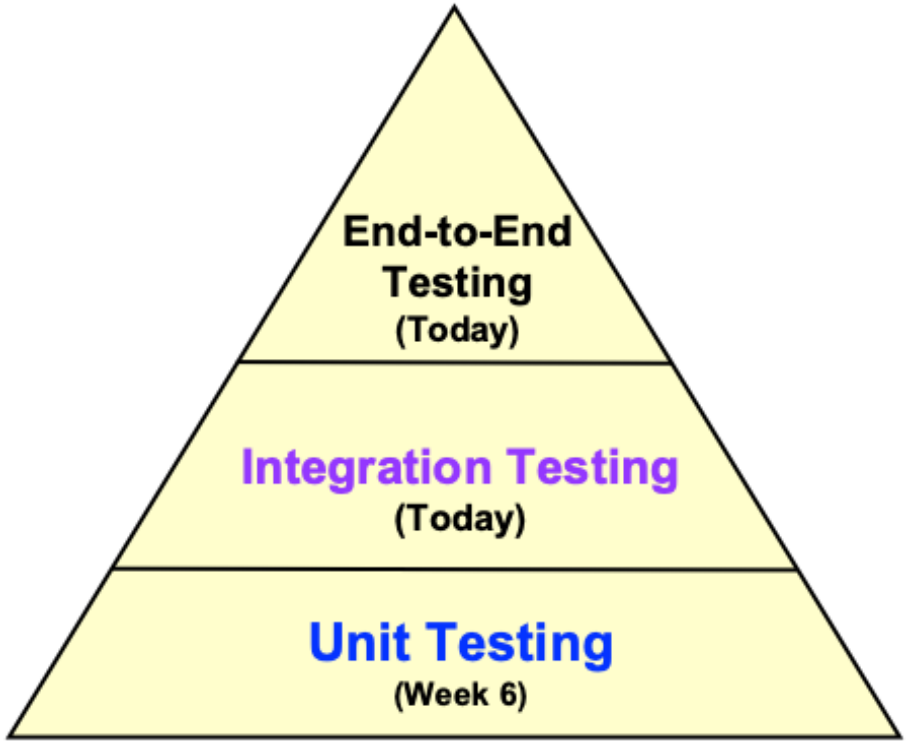
Outline

- ***Part 1 – Test Doubles*** – stubs, mocks, fakes; `unittest.mock`
- ***Part 2 – Integration Testing*** – real components, controlled boundary; test data
- ***Part 3 – End-to-End & Flaky Tests*** – Playwright; test agents; flakiness
- ***Part 4 – Continuous Delivery*** – pipelines, release gates, other evidence

The Question for Today

Today – Does the system behave as expected <u>across boundaries</u>?
Test doubles · integration · E2E · flaky tests → release gates

Week 6 – Does <u>each component</u> behave as expected?
Unit test design · coverage · TDD · CI as the gate on a pull request



SMU Classification: Restricted

Test Doubles

Before we move up the pyramid…
how do we test code that depends on something external?

Test Doubles

- A test double is a substitute for a real dependency, used during testing
- It lets us control what the dependency does and/or observe how our code interacts with it
- What we replace depends on the scope of the test
  - ***Unit test:*** *often replace significant dependencies to isolate one unit*
  - ***Integration test:*** *keep collaborating components real; double only selected boundaries*
  - ***End-to-end test:*** *use as few doubles as practical*



- A “double” is the general idea; stub, mock, fake, etc. describe different roles a double can play. We will explore some using `unittest.mock`

Our Unit Under Test – `EWallet.top_up()`

```python
# wallet.py
from notifications import send_receipt

class EWallet:
    def __init__(self, owner, balance, gateway):
        self.owner = owner
        self.balance = balance
        self.gateway = gateway                # passed in

    def top_up(self, amount, card_token):
        if amount <= 0:
            return False
        result = self.gateway.charge(card_token, amount)
        if not result.approved:
            return False
        self.balance += amount
        send_receipt(self.owner, amount)      # imported
        return True
```

① gateway – passed in

Injected dependency that ultimately relies on an external payment API

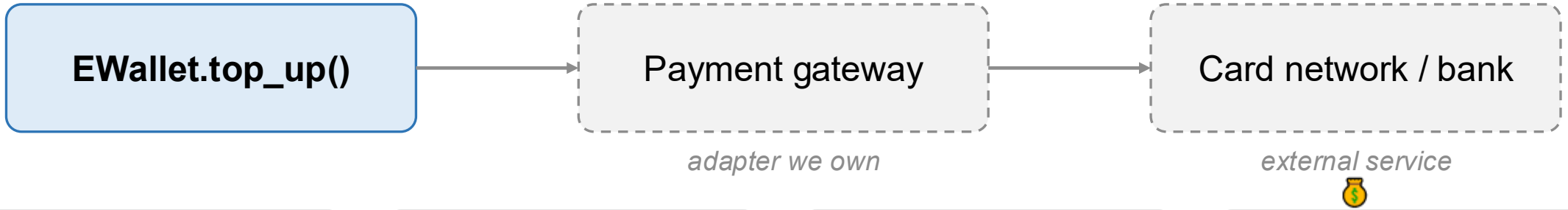
② send_receipt – imported

Imported dependency called directly from inside `top_up()`

③ What should unit tests for `top_up()` actually check?

Its observable outcomes: the return value, the balance, and whether exactly one receipt is sent – not the internal workings of its dependencies

Why Isolate a Unit?



Slow
A network round trip in every test

Costs money
Real charges, fees and rate limits

Unpredictable
Timeouts, outages, changing responses

Hard to trigger
How do you make a real card decline on demand?

How do we test `top_up()` **without** charging a real card?

Test Doubles – A Map

Double	What it does	Example
Dummy	Fills a parameter; never actually used	Any gateway, passed to `top_up(-5)` – which rejects the input before calling the gateway
Stub	Returns canned answers	A gateway whose charge() method always returns a declined payment, regardless of the input
Mock	Checks that the expected calls were actually made	A mock receipt object that verifies `send_receipt("alice", 50)` was called exactly once.
Fake	A working, simplified implementation	An in-memory gateway with card limits

Which double(s) should we use when unit testing `top_up()`?

Stubs vs Mocks – What are you Checking?

State verification (via stub)

Control what the dependency returns, then check your code's result, e.g.

gateway declines
→ `assertFalse(result)`; balance still 100

Behaviour verification (via mock)

Check that an interaction happened, with the right arguments, e.g.

top-up approved
→ `send_receipt` called once with `("alice", 50)`

When should we use **stubs** vs. **mocks**?

unittest.mock – Stubbing the Gateway

```python
import unittest
from unittest.mock import Mock
from wallet import EWallet

class TestTopUp(unittest.TestCase):

    def setUp(self):
        self.gateway = Mock()        # a stand-in for the gateway
        self.wallet = EWallet("alice", 100, self.gateway)

    def test_declined_card_leaves_balance_unchanged(self):
        # stub: a canned answer – this card is declined
        self.gateway.charge.return_value = Mock(approved=False)

        result = self.wallet.top_up(50, "tok_visa")

        self.assertFalse(result)
        self.assertEqual(self.wallet.balance, 100)
```

Mock()

Accepts any call; returns another Mock object

.return_value Turns `self.gateway.charge()` into a **stub**, i.e. a canned answer (approved=False)

Assert on state

i.e. check the result and balance is correct when a charge is declined

Caution! ⚠ *In `unittest.mock`, a Mock object can play different roles.*
*Here, it acts as a **stub** because we **only** configure a canned return value **(approved=False)***

unittest.mock – "Patching" in a Mock

```python
from unittest.mock import Mock, patch

class TestTopUp(unittest.TestCase):
    # ... setUp() as before ...

    @patch("wallet.send_receipt")      # replaced for this test only
    def test_approved_top_up_sends_one_receipt(self, mock_send):
        self.gateway.charge.return_value = Mock(approved=True)

        result = self.wallet.top_up(50, "tok_visa")

        self.assertTrue(result)                          # state
        self.assertEqual(self.wallet.balance, 150)       # state
        mock_send.assert_called_once_with("alice", 50)   # behaviour
```

⚠ *Without the patch, `top_up()` would call the real `send_receipt()`*

① @patch(…)

Replaces `wallet.send_receipt` with a Mock

② mock_send

patch also passes that Mock into the test as the **mock_send** parameter

③ assert_called_once_with(…)

The assertion checks that the Mock was called **once** with the given arguments

Gotcha – Patch Where the Name Is Looked Up

```
# wallet.py
from notifications import send_receipt   # wallet's own name
```

✗ **`@patch("notifications.send_receipt")`**

Replaces the name inside notifications, but wallet.py still holds the original.
A real e-mail is sent, and the Mock is never called.

✓ **`@patch("wallet.send_receipt")`**

Replaces the name where top_up() actually looks it up.

Patch the Mock into the name where it is used, **not** where it is defined

Gotcha – What's Wrong with this Test…?

An agent was asked to *"write passing tests for EWallet.top_up()"*. It produced:

```
@patch("wallet.EWallet.top_up")
def test_top_up_succeeds(self, mock_top_up):
    mock_top_up.return_value = True
    wallet = EWallet("alice", 100, Mock())

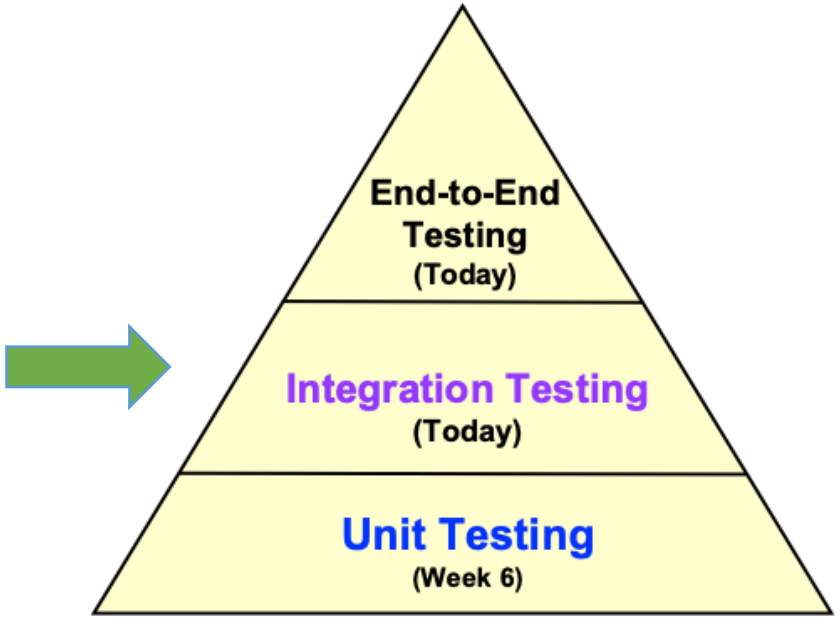
    self.assertTrue(wallet.top_up(50, "tok_visa"))
    mock_top_up.assert_called_once_with(50, "tok_visa")
```

What's **problematic** about this test? 👀

SMU Classification: Restricted

Integration Testing

real components together;
controlled external boundary



- Multiple components (modules) of a system are **tested together**
- For example, “does placing an **order** correctly reduce the remaining **inventory**?”
- Do the components correctly integrate with the DB?

The Mock-Based Integration Illusion

EWallet tests ✅

Gateway stubbed.

Author assumes `charge(card, 50)` means **$50**

?

Gateway adapter tests ✅

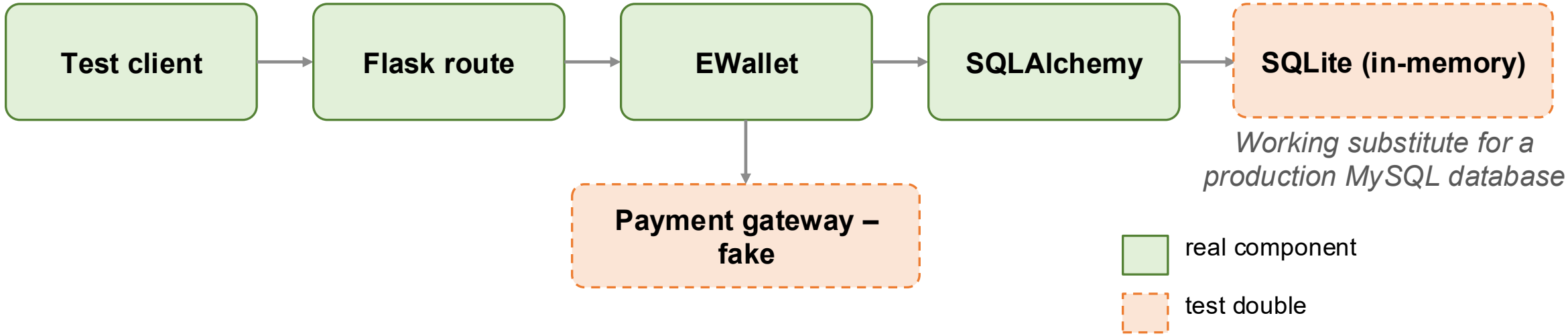
Vendor SDK stubbed.

Author assumes amounts are in **cents**

In production: `top_up(50)` charges 50 cents ❌

Unit tests can validate each side in isolation and still miss a broken boundary.
***Integration tests** test the **boundary itself**.*

Integration Tests in Flask – What Is Real?



- ***Exercise the real application path:*** route → business logic → ORM
- ***Use isolated test infrastructure:*** a fresh in-memory SQLite database for each test
- ***Replace only external dependencies:*** e.g. the payment gateway
- ***Remember the trade-off:*** SQLite is convenient, but may not expose MySQL-specific behaviour

```python
class FakeGateway:
    """In-memory stand-in: real behaviour, no network."""
    def __init__(self, limits):
        self.limits = dict(limits)    # card -> limit

    def charge(self, card_token, amount):
        approved = self.limits.get(card_token, 0) >= amount
        if approved:
            self.limits[card_token] -= amount
        return Result(approved=approved)
```

Worked Example (1/2) – Real App, Test DB, Fake Gateway

```python
class TestTopUpIntegration(unittest.TestCase):

    def setUp(self):
        self.gateway = FakeGateway({"tok_visa": 500})      # fake gateway
        self.app = create_app({
            "TESTING": True,
            "SQLALCHEMY_DATABASE_URI": "sqlite:///:memory:", # test DB
            "PAYMENT_GATEWAY": self.gateway,
        })
        self.client = self.app.test_client()
        with self.app.app_context():
            db.create_all()
            db.session.add(Wallet(owner="alice", balance=100))
            db.session.commit() # above EWallet ($100) available for every test

    def tearDown(self):
        with self.app.app_context():
            db.drop_all()
```

Real

Flask app, route, EWallet, SQLAlchemy

Doubles

Payment gateway + in-memory SQLite database

Fresh state

The fixture creates and tears down a fresh database for **every** test

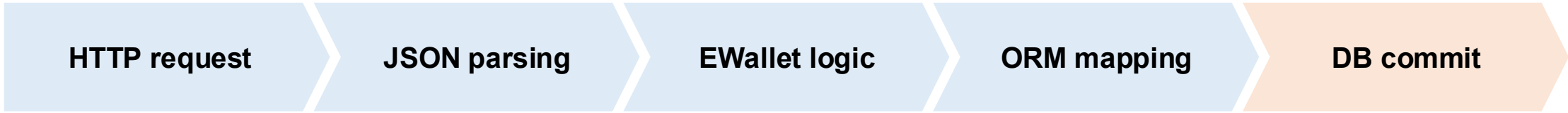
⚠ *`create_app({...})` is an application setup function defined by our application. It lets us create the app with test-specific settings.*

Worked Example (2/2) – Through the Route to the Database

```
    def test_top_up_persists_balance_and_transaction(self):
        resp = self.client.post("/wallets/alice/top-up",
                                json={"amount": 50, "card_token": "tok_visa"})

        self.assertEqual(resp.status_code, 200)
        with self.app.app_context():
            wallet = Wallet.query.filter_by(owner="alice").one()
            self.assertEqual(wallet.balance, 150)
            self.assertEqual(Transaction.query.count(), 1)
```

One test, through the real application path:



Test Data

- Start every test from a **known state**: fresh data per test (FIRST: *Isolated*, *Repeatable*)
- Build test data with small **helper functions**, not one giant shared fixture
- Make the values that matter **visible in the test**; default the rest
- Never use **raw production personal data** in tests

```python
def make_wallet(owner="alice", balance=0):
    wallet = Wallet(owner=owner, balance=balance)
    db.session.add(wallet)
    db.session.commit()
    return wallet
```

```python
# in a test: only what
# matters is visible
make_wallet(balance=100)
```

Aside – When Doubles Aren't Enough

Contract tests (e.g. Pact)

Check that your double and the real service agree on the **contract** — schema, fields, formats, units, status codes

The consumer records what it **expects**; the provider verifies it still **satisfies those expectations**

Would have caught dollars vs cents

Disposable real infrastructure

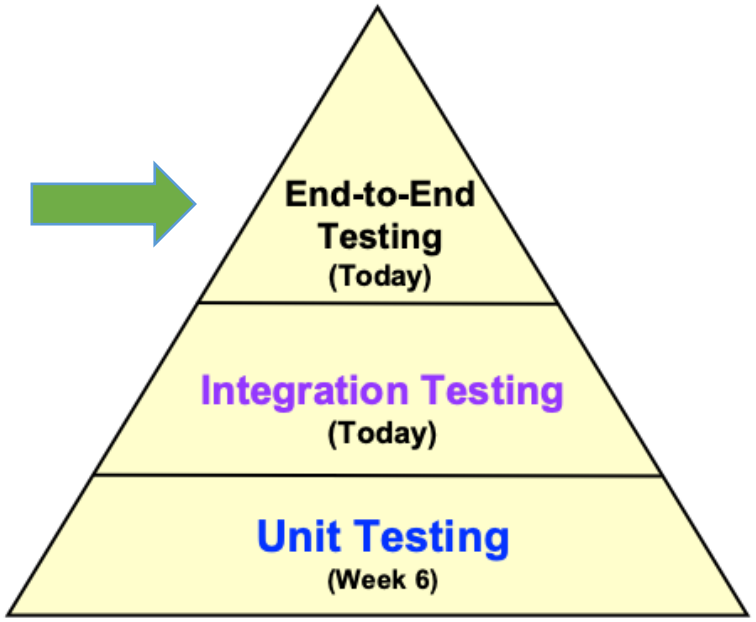
Start the real database (or message broker) in a throwaway container for the test run, e.g. **Testcontainers**, or Docker containers

Use when "SQLite ≠ our production database" matters: SQL dialect, constraints, concurrency

Use doubles for speed; check them against reality

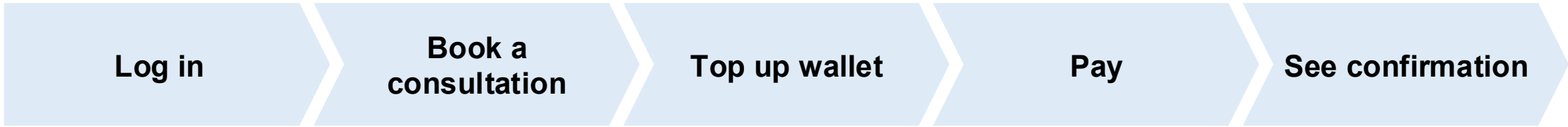
End-to-End Testing

the whole system,
through the user's eyes 👀



- Test the **integrated flow** of the application from start to finish, simulating real user scenarios
- Verify that all subsystems work together correctly

What End-to-End Tests Verify



- Drive the **running app** through a **real browser**, as a user would
- Validates the **requirement** – not the code
- Slow and brittle → keep **a few**, for the **critical journeys**

Which few E2E journeys are critical enough to run as **smoke tests** on every build?

Playwright

- **Real browsers** – Chromium, Firefox, WebKit
- **Python API** (also JavaScript/TypeScript, Java, .NET)
- **Locators by role, label and text** – how a user finds things, not brittle CSS paths
- **Auto-waiting** – waits until an element is ready before acting on it
- **Codegen** records your clicks as a test; the **trace viewer** replays a failed run step by step

Demo! 

Example – Booking and Paying for a Consultation

```python
from playwright.sync_api import expect

class TestBookingJourney(unittest.TestCase):
    # setUp() launches a browser: self.page = browser.new_page()

    def test_student_can_book_and_pay(self):
        page = self.page
        page.goto("http://localhost:5000/login")
        page.get_by_label("Email").fill("alice@example.com")
        page.get_by_label("Password").fill("test-password")
        page.get_by_role("button", name="Log in").click()

        page.get_by_role("link", name="Book a consultation").click()
        page.get_by_label("Topic").select_option("Data Analytics")
        page.get_by_role("button", name="Pay with e-wallet").click()

        expect(page.get_by_text("Booking confirmed")).to_be_visible()
```

Locators

Find elements by role, label, or visible text rather than brittle CSS paths

expect(...)

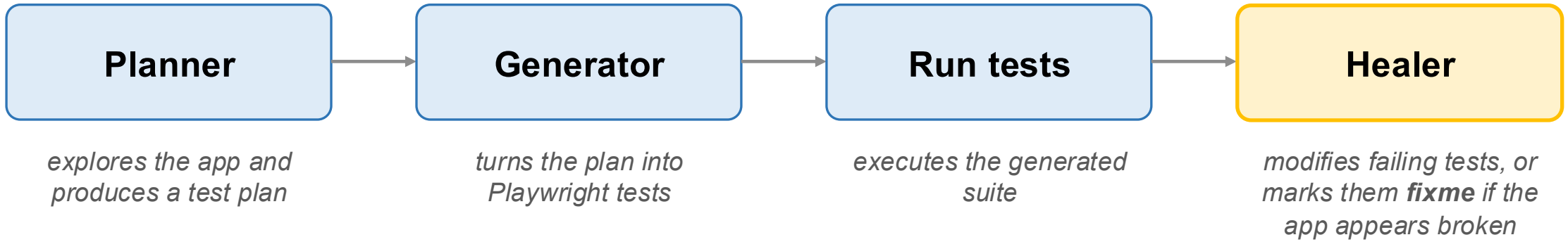
Waits for the expected condition to become true before failing

Expected outcome

Defines what success means – here, that “Booking confirmed” becomes visible

Playwright Test Agents

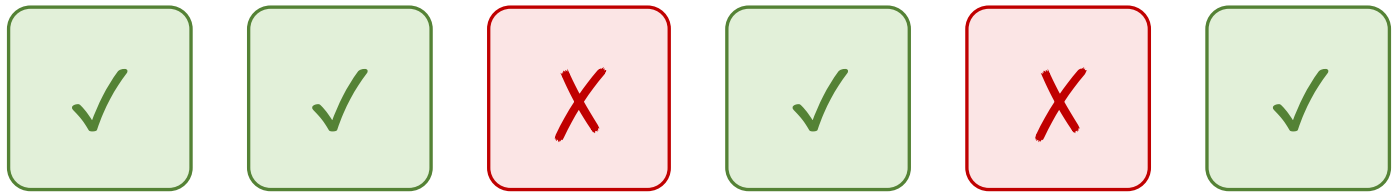
- Playwright now ships three agent definitions that plug into a coding agent…
- …and a "healer" that runs failing tests, inspects the UI, and patches the test



What does the **healer** have to infer?

Flaky Tests

A **flaky test** passes and fails on the **same code**, with no changes



same commit – six CI runs

- Worse towards the top of the pyramid (E2E): browser, network, timing, shared data
- The real cost is **trust**: red starts to mean “just re-run it” – and real failures get ignored

Common Causes of Flakiness

⏱ Timing

Clicking before the page has finished loading; a fixed `sleep()` too short on a slow CI runner

🔗 Test order & shared state

Test B relies on data that test A left behind

📅 Time & date

Passes all day; fails at midnight, at month-end or on 29 February

🎲 Randomness

Random IDs, shuffled data, unseeded generators

🌐 External services

Real network calls: timeouts, rate limits, outages

⚙ Concurrency & resources

Race conditions, ports already in use, leaked connections

Fixing Flakiness – and How Not To

✗ Anti-fixes

Arbitrary `sleep(5)` to handle timing issues

Retry until green

Delete or weaken the assertion

Skip it, and forget it

✓ Real fixes

Wait for a **condition** rather than a duration

Inject the clock; seed randomness

Fresh state for every test

Double the network boundary

Where the Slow Tests Run **(Week 6 Recall)**

1 Your machine, and your agent's loop

Unit tests and fast integration tests – seconds

2 CI on the pull request

The required gate: unit + integration tests, plus a few E2E smoke tests – minutes

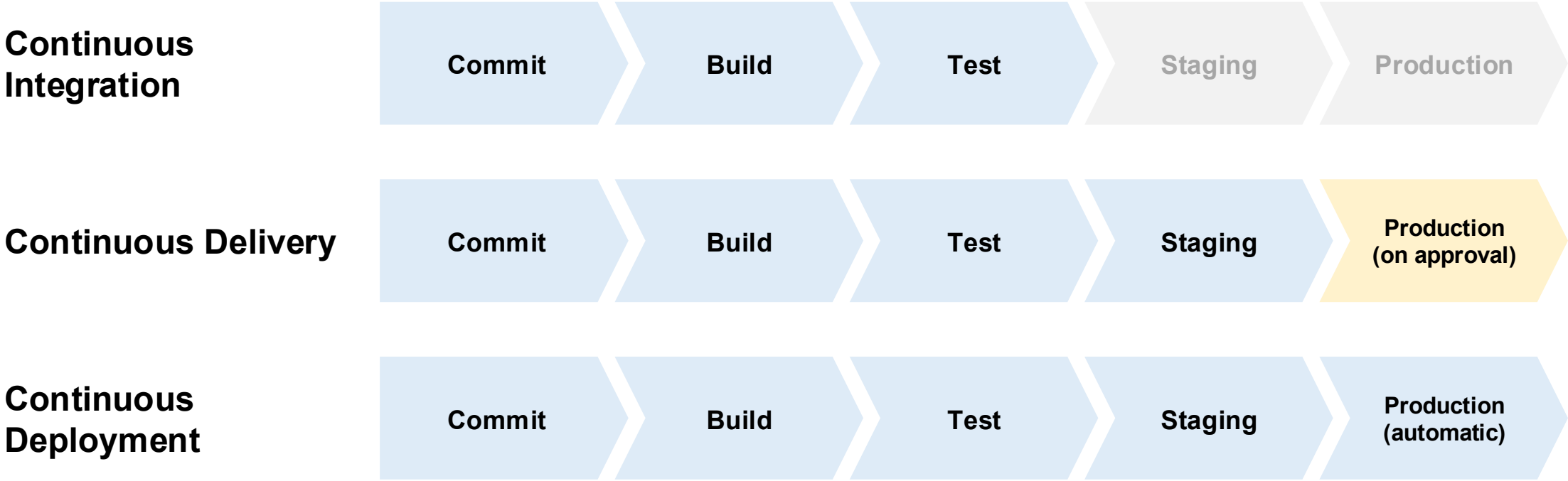
3 Scheduled and post-merge runs

The full E2E suite across browsers, slower checks (e.g. real-database containers) – and a good place to spot flaky tests

Continuous Delivery

from a green build ✅ ✅ ✅
to a safe release

CI → Continuous Delivery → Continuous Deployment



Continuous delivery: every change is kept ***releasable***; releasing is a business decision

Continuous deployment: every change that passes the pipeline ***is*** **released**

A Possible CI/CD Pipeline for a Flask Application



e.g. package as a .zip file

Test the source, then test what you will actually deploy.

Promote the same artifact through staging and production, with a human approval gate.

Gating a Release

- ✓ **Required checks** are green – unit and integration tests **(Week 6 & 9)**
- ✓ Package passes **smoke tests** and is deployed to staging **(Today)**
- ✓ An **accountable person approves** the production release (continuous delivery)
- ✓ **Production credentials** are available only to the production deploy job
- ✓ After deploying: **smoke-test production** – roll back automatically if it fails

In GitHub Actions – Environments

```yaml
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    # ... unit + integration tests

  deploy-staging:
    needs: test
    runs-on: ubuntu-latest
    environment: staging
    # ... deploy, then run E2E smoke tests

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production    # rule: required reviewers
    # ... deploy the same artifact
```

environment:

Links a job to that environment's protection rules

Required reviewers

Configured under Settings → Environments → production; the job waits for approval before running

Environment secrets

Only jobs for that environment can read them

Demo!

Other Evidence a Release Gate Can Require

Dependency review

Does this change add a dependency with a known vulnerability?

Secret scanning

Did someone commit a password or an API key?

Static analysis

Linting and security-focused checks on the code itself

SBOM

A software bill of materials: an inventory of everything inside the artifact **(see Week 9 Lab!)**

SMU Classification: Restricted

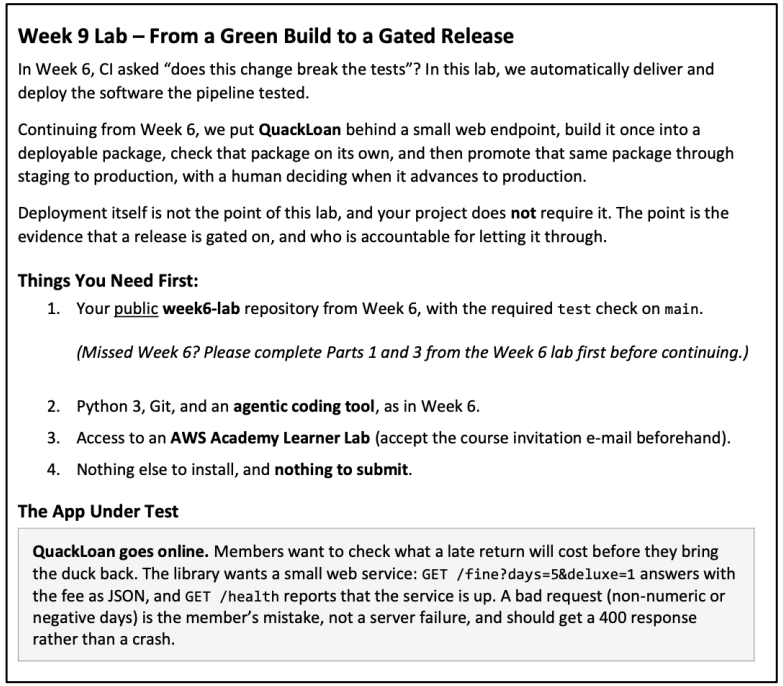
Lab Exercise

Lab Exercise – CD with GitHub Actions

- Access the **Week 9 Lab Exercise** in eLearn

- Extends the Week 6 lab to a CI/CD pipeline
  - *Packages the release as a .zip*
  - *Generates an SBOM*

- Demonstrates an automated deployment to AWS
  - *We will use an AWS Learner Lab (no credit card required)*
  - ***NB:** deployment is not required for the project*



Week 9 Lab – From a Green Build to a Gated Release

In Week 6, CI asked "does this change break the tests"? In this lab, we automatically deliver and deploy the software the pipeline tested.

Continuing from Week 6, we put **QuackLoan** behind a small web endpoint, build it once into a deployable package, check that package on its own, and then promote that same package through staging to production, with a human deciding when it advances to production.

Deployment itself is not the point of this lab, and your project does **not** require it. The point is the evidence that a release is gated on, and who is accountable for letting it through.

Things You Need First:

1. Your public **week6-lab** repository from Week 6, with the required `test` check on `main`.

   (Missed Week 6? Please complete Parts 1 and 3 from the Week 6 lab first before continuing.)

2. Python 3, Git, and an **agentic coding tool**, as in Week 6.
3. Access to an **AWS Academy Learner Lab** (accept the course invitation e-mail beforehand).
4. Nothing else to install, and **nothing to submit**.

The App Under Test

QuackLoan goes online. Members want to check what a late return will cost before they bring the duck back. The library wants a small web service: `GET /fine?days=5&deluxe=1` answers with the fee as JSON, and `GET /health` reports that the service is up. A bad request (non-numeric or negative days) is the member's mistake, not a server failure, and should get a 400 response rather than a crash.

Aside – Continuous Delivery in the Project

- Continuous Delivery is presented for your learning and to show how testing and CI lead to a releasable application

- It is in-scope for quizzes and the exam, but **not** for your project

- Your project's pipeline should focus on Continuous Integration
  - *Extending to CD isn't expected; you do **not** need to assemble zip after zip!*
  - *It doesn't score extra marks* 😊

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 😊
- **http://smu.sg/is212-reflections-2026**

